

PowerShell Incident Data Response Collector

Trenton Carter – COMP 325, 10/16/2025

Overview

During an active security incident, responders must quickly gather system and process information to understand what is happening on a host. Manually collecting this data is slow and error prone.

In this lab, I developed a PowerShell script called `IncidentResponseCollector.ps1` that automates basic incident response data collection. The script creates an output directory, captures high-value system information, exports detailed process data, and generates a human-readable incident summary. This simulates a lightweight IR collection tool that an analyst could run early in an investigation to snapshot the system state.

Execution

1. Script Initialization

The script defines a function called *Invoke-IncidentResponse* that accepts one parameter: the output path where all collected artifacts will be stored. If the directory does not exist, the script automatically creates it. This ensures that evidence is organized in a single location and prevents overwriting other files.

```
PS C:\> cat IncidentResponseCollector.ps1
# Incident Response Data Collection Script
function Invoke-IncidentResponse {
    param(
        [string]$OutputPath = "./incident_data"
    )

    # Create output directory
    if (-not (Test-Path $OutputPath)) {
        New-Item -ItemType Directory -Path $OutputPath -Force | Out-Null
    }
}
```

2. System Information Collection

The script collects important system details including Timestamp of collection, Hostname, Username executing the collection, PowerShell version, OS information, Number of running processes, and simulated uptime. This information is stored in a data structure and then written to `system_info.json` in a readable JSON format. The JSON file acts as a structured snapshot of the host during the time of the incident.

```

Write-Host "=== Incident Response Data Collection ==="
Write-Host "Output Path: $OutputPath"

# Collect system information
$SystemInfo = @{}
    'Timestamp'      = Get-Date
    'Hostname'       = $env:HOSTNAME
    'User'           = $env:USER
    'PowerShell Version' = $PSVersionTable.PSVersion
    'OS Info'        = $PSVersionTable.OS
    'Process Count'   = (Get-Process).Count
    'Uptime'         = (Get-Date) - [datetime]"2024-08-05 08:00:00" # Simulated boot time
}

# Save system information
$SystemInfo | ConvertTo-Json | Out-File -FilePath "$OutputPath/system_info.json"
Write-Host "[+] System information collected"

```

3. Process Information Collection

Next, the script gathers a detailed list of all running processes. For each process it records: Process name, Process ID (PID), CPU usage, Memory usage (WorkingSet), Start time, Executable name. The full process list is exported to processes.csv, which can be opened in spreadsheet tools for analysis. This data helps identify suspicious executables, abnormal resource usage, and unusual long-running processes.

```

# Collect process information
$ProcessInfo = Get-Process | Select-Object Name, Id, CPU, WorkingSet, StartTime, ProcessName
$ProcessInfo | Export-Csv -Path "$OutputPath/processes.csv" -NoTypeInformation
Write-Host "[+] Process information collected"

```

4. Generating the Incident Summary

Finally, the script creates a text file called incident_summary.txt. This file includes: Collection date and time, Hostname, User who ran the script, A list of generated evidence files, Recommendations for initial triage steps. These recommendations include reviewing suspicious processes, checking for unexpected reboots, and correlating timestamps with security event logs. This summary simulates what an analyst would review at the start of an investigation.

```

# Create incident summary
$Summary = @{}
Incident Response Summary
=====
Collection Date: $(Get-Date)
Hostname: $($env:HOSTNAME)
Collected by: $($env:USER)

Files Generated:
- system_info.json: System configuration and status
- processes.csv: Running process details
- incident_summary.txt: Collection activity log

Recommendations:
- Review process list for suspicious executables
- Check system uptime for unexpected reboots
- Correlate timestamps with security events
"@

$Summary | Out-File -FilePath "$OutputPath/incident_summary.txt"
Write-Host "[+] Incident summary created"
Write-Host "Data collection completed successfully!"
}

# Execute incident response collection
Invoke-IncidentResponse

```

```
(kali㉿kali)-[/home/kali/cybersec-labs/powershell-lab/SecurityAnalysis]
PS> ./IncidentResponseCollector.ps1
Incident Response Data Collection
Output Path: ./incident_data
[+] System information collected
[+] Process information collected
[+] Incident summary created
Data collection completed successfully!
```

Findings & Results

Running the incident response collector produced a set of organized forensic artifacts. These files captured key system information and a full process list that could be analyzed for abnormal or malicious activity. The summary file also helped document the collection event and guide next steps.

Although this script is simple, it demonstrates how automation can improve the accuracy and speed of incident response. It reduces the chance of missing important system details during early investigation stages.

Skills Demonstrated

- PowerShell scripting and function creation
- Directory and file automation
- JSON and CSV data export
- Process enumeration
- System information collection
- Evidence preparation for IR workflows
- Writing structured summaries for analysis

Conclusion

This lab strengthened my understanding of how PowerShell can be used in cybersecurity for rapid data collection during an incident. By automating the capture of system information, running processes, and environmental data, the script ensures reliable and organized forensic evidence. This type of automation is extremely valuable in real-world incident response, where speed and consistency are critical.